Lab 3: Local search

Authors: Piotr Foltyniewicz 160295, Jakub Adamski 160291

Problem description

Each instance consists of 200 nodes. A row in the input file contains the node's planar coordinates and selection cost, in the form `x;y;cost`. Node indices are zero-based and correspond to row order. The task is to select exactly 100 distinct nodes (half of the available nodes), order them, and form a closed cycle that minimizes the sum of travel distances and selected-node costs.

Distances are Euclidean, rounded to the nearest integer and precomputed after reading the instance. All algorithms use this distance matrix. The evaluation sums all selected-node costs and all cycle edges, including last -> first. In the pseudocode, `m = 100`, `n = 200`, `c[v]` is node cost, and `d(a, b)` is the precomputed distance. A solution `S` is an ordered list of `m` distinct nodes; `U` denotes the set of nodes not in `S`.

The objective for an ordered selection `S` is:

```
F(S) = sum(c[S[i]]) + sum(d(S[i], S[(i + 1) mod length(S)]))
```

Pseudocode

All indices are zero-based. Local search starts from a given solution and applies moves until none improves the objective. Moves are evaluated only through deltas; the neighbor solution is never built and `F` is never recalculated from scratch. The eight methods combine: steepest or greedy search, node or edge exchange as the intra-route move, random or greedy starting solution.

Neighborhood and deltas

The neighborhood consists of inter-route moves and one kind of intra-route moves. A move is only a descriptor (type and positions).

- `INTER(p, v)`: replace the selected node at position `p` with an unselected node `v`.
- `SWAP_NODES(i, j)`, `i < j`: exchange the nodes at positions `i` and `j`.
- `SWAP_EDGES(i, j)`, `i < j`: replace edges `(S[i], S[i+1])` and `(S[j], S[j+1])` with `(S[i], S[j])` and `(S[i+1], S[j+1])` by reversing `S[i+1..j]`. Edges sharing a node are skipped.

Positions are taken modulo `m`, so the closing edge is handled like the others.

```
DELTA_INTER(S, p, v):
    out = S[p]; prev = S[p - 1]; next = S[p + 1]
    return d(prev, v) + d(v, next) - d(prev, out) - d(out, next) + c[v]
- c[out]
```

```
DELTA_SWAP_EDGES(S, i, j):
    a = S[i]; b = S[i + 1]; e = S[j]; f = S[j + 1]
    return d(a, e) + d(b, f) - d(a, b) - d(e, f)

DELTA_SWAP_NODES(S, i, j):
    E = distinct edge starts among {i - 1, i, j - 1, j}
    return sum over e in E of d'(e, e + 1) - d(e, e + 1)     // d' =
edge length after the swap
```

Steepest local search

```
STEEPEST(S):
    loop:
        best = NONE; bestDelta = 0
        for move in all moves:
            delta = DELTA(S, move)
            if delta < bestDelta: best = move; bestDelta = delta
        if best == NONE: return S
        APPLY(S, best)
```

Greedy local search

Randomization. All moves of both kinds (inter-route and the chosen intra-route kind) form one set of move descriptors. Moves are taken from it in uniformly random order: each time a move is needed, a random not yet examined move is drawn, so the two kinds are mixed and the search does not start preferentially from either of them. Deltas are not precomputed: only the drawn move is evaluated, and the first move with a negative delta is applied immediately. Therefore only as many moves are evaluated as are needed to find an improvement (all of them only at a local optimum). After a move is applied, the set of moves is generated again. The random generator is `std::mt19937` seeded from `std::random_device`.

```
GREEDY(S):
    loop:
        R = all moves                                  // both kinds
        improved = false
        while R is not empty:
            move = random element removed from R   // uniformly random
order
            if DELTA(S, move) < 0:
                APPLY(S, move)
                improved = true
                break
        if not improved:
            return S
```

Starting solutions

Random: the first `m` nodes of a random permutation of all `n` nodes. Greedy: the best construction heuristic of labs 1-2 for the instance, started from each of the 200 nodes: weighted-sum greedy cycle for TSPA and weighted-sum nearest neighbor for TSPB.

Each method was run 200 times per instance (200 random solutions, or each node as the construction start). Times include the construction of the starting solution.

Results

Each of the eight methods was run 200 times on each instance.

Objective values use the format **average (minimum – maximum)**; running times are averages per run in milliseconds. Lower objective values are better.

Objective function values

Method	TSPA	TSPB
Steepest Nodes Random	90292.47 (84621 – 99661)	65902.04 (60136 – 75295)
Steepest Nodes Greedy	73021.71 (**71955** – 74861)	54292.07 (49978 – 61265)
Steepest Edges Random	76108.17 (73719 – 79105)	52640.92 (50335 – 55286)
Steepest Edges Greedy	73003.80 (**71955** – 74861)	53091.28 (**49346** – 60354)
Greedy Nodes Random	88266.88 (82357 – 98469)	63746.16 (59205 – 70900)
Greedy Nodes Greedy	73019.87 (**71955** – 74877)	54310.27 (49913 – 61265)
Greedy Edges Random	76299.05 (73468 – 80279)	53038.72 (50575 – 55602)
Greedy Edges Greedy	72984.73 (**71955** – 74861)	53251.93 (49660 – 60266)
Greedy Cycle 2-regret heuristics (lab 2)	120378.77 (112806 – 128089)	72484.84 (68150 – 77274)
Greedy Cycle heuristics with a weighted sum criterion (lab 2)	73353.51 (**72011** – 75532)	57866.33 (52731 – 63370)
NN heuristics 2-regret heuristics (lab 2)	121360.29 (110950 – 135771)	73649.84 (68761 – 79756)
NN heuristics with a weighted sum criterion (lab 2)	75561.20 (72419 – 78504)	55698.34 (**50485** – 61406)

Method	TSPA	TSPB
Random (lab 1)	261559.68 (234022 – 287462)	205914.49 (183676 – 233336)
Nearest Node End (lab 1)	87476.63 (84362 – 90075)	59397.34 (56007 – 63166)
Nearest Node Any (lab 1)	74306.53 (**72328** – 77022)	59476.23 (**52915** – 66164)
Greedy Cycle (lab 1)	73743.73 (72590 – 75281)	59650.81 (55655 – 63734)

Running times (ms)

Total times cover the complete run: creating the starting solution and performing local search.

Method	TSPA: total	TSPA: starting solution	TSPB: total	TSPB: starting solution
Steepest Nodes Random	67.694814	0.003531	66.661623	0.003519
Steepest Nodes Greedy	5.657000	4.066111	8.942153	4.111100
Steepest Edges Random	35.005239	0.003531	35.416348	0.003519
Steepest Edges Greedy	4.856945	4.066111	9.898119	4.111100
Greedy Nodes Random	28.357479	0.003531	28.458643	0.003519
Greedy Nodes Greedy	5.509622	4.066111	6.848713	4.111100
Greedy Edges Random	24.942397	0.003531	24.829678	0.003519
Greedy Edges Greedy	5.153622	4.066111	8.615719	4.111100
Greedy Cycle 2-regret heuristics (lab 2)	4.517217		3.915432	
Greedy Cycle heuristics with a weighted sum criterion (lab 2)	4.066111		3.760381	
NN heuristics 2-regret heuristics (lab 2)	4.025403		4.062423	
NN heuristics with a weighted sum criterion (lab 2)	4.148715		4.111100	
Random (lab 1)	0.030866		0.011238	
Nearest Node End (lab 1)	0.142488		0.078278	
Nearest Node Any (lab 1)	4.214506		3.637342	
Greedy Cycle (lab 1)	3.036610		2.781263	

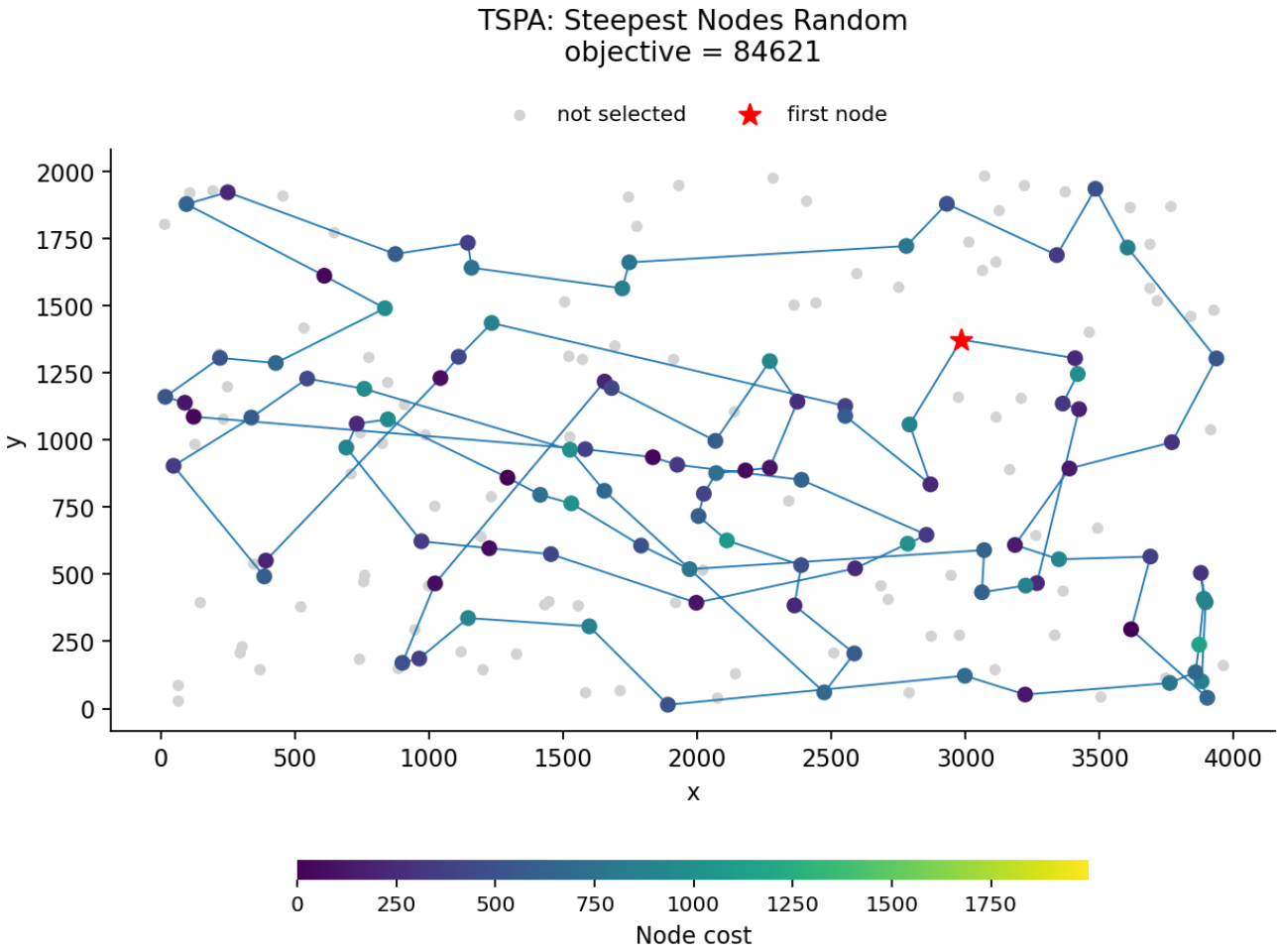
Best solutions and visualizations

Node color represents selection cost; the compact horizontal scale below each plot uses the same cost range for all methods on an instance. Gray nodes are not selected, blue segments form the closed tour, and the red star marks the first node in the displayed sequence. Lists below show ten node indices per row; read left to right, then top to bottom to follow the tour.

TSPA

Steepest Nodes Random

Objective: **84621**.

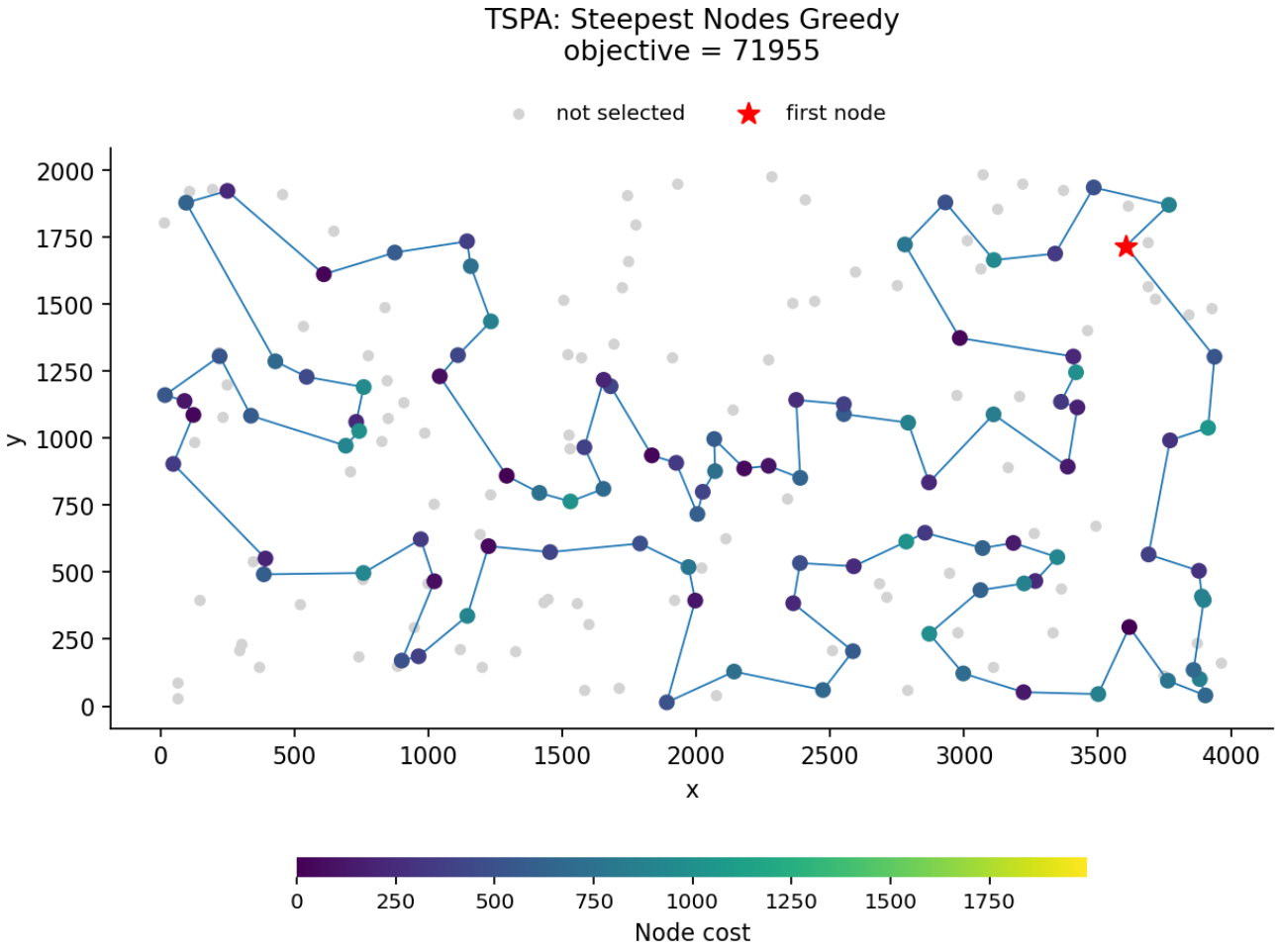


Solution (tour order):

```
164   1 169   3  50  89  42 151 150  73
 45 102  28 180  90 123 160  30 141  87
135 134  72 176 131 144  67  35  63 149
116  60 108  38  80  31  48  91 157  71
139 112 186 136 178  29 101 147  47  93
106 113 130 107  78 193 170 159 146  75
153  88  57 182 114  98 189   2 171 187
 51  16  52 143  33  36 177 179  53 140
 41  84  58 158 192 185 156 100 152  49
168 127  97 132  76   4   9  54 174   8
```

Steepest Nodes Greedy

Objective: **71955**.



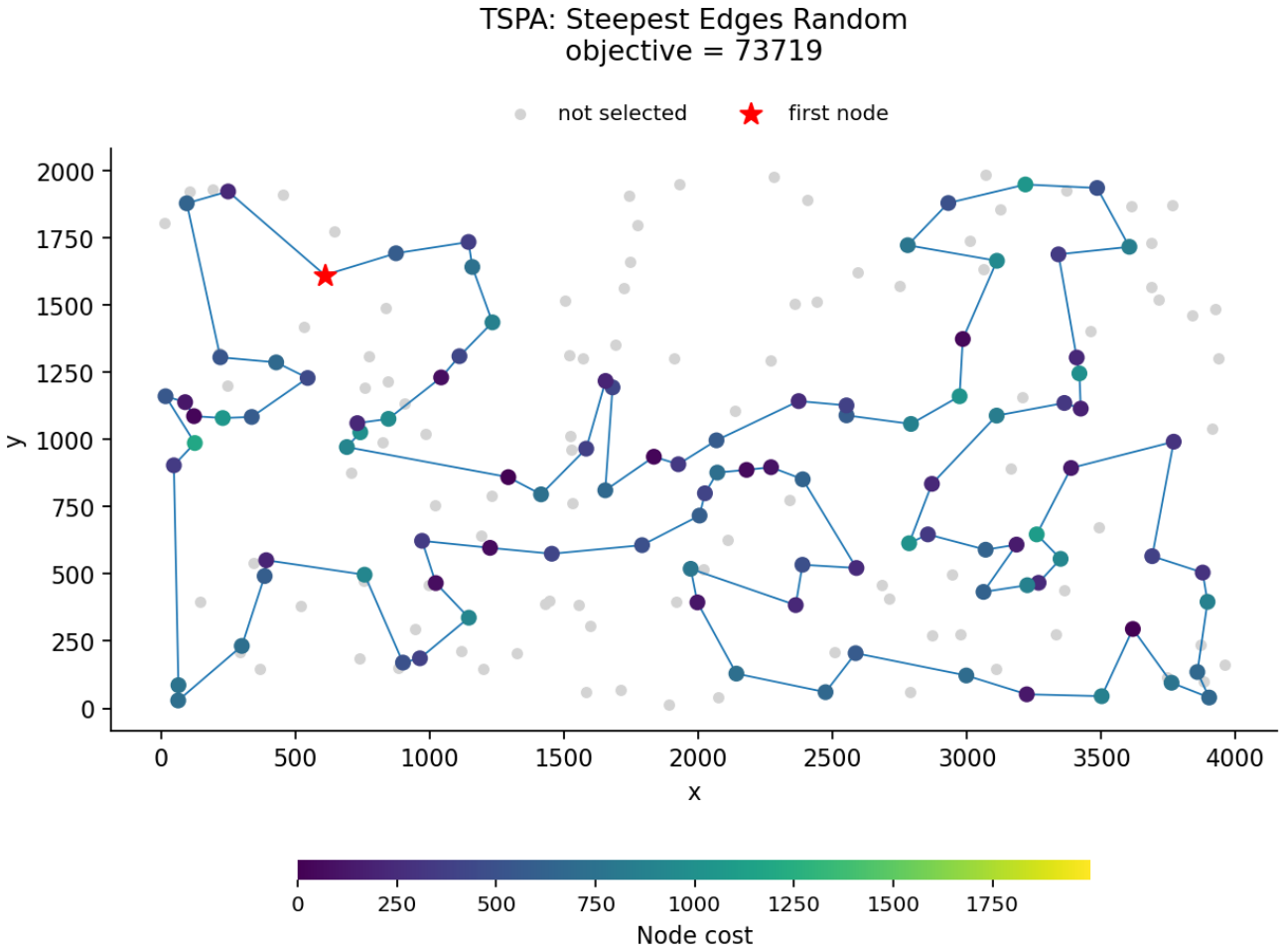
Solution (tour order):

```
101  55  29 178 184 136 186 164   1 169
  3  50  93  85 174   8  54   9  33 131
 36 177  52 179  53 140 144  67  16  51
 35 185 102  28 180 132  76   4  71 157
 91  80  48  31 108 152 100 123  83 160
 49  60 116 149  63 168  97 127 161  30
```

```
187 171   2 189 141  87  45  73 135 114
181 192 158  58  84 134  72 176 150 106
113  89  42 151 138 182  57  64 107  88
 78 193 153 170 146 159 130  47  37 147
```

Steepest Edges Random

Objective: **73719**.

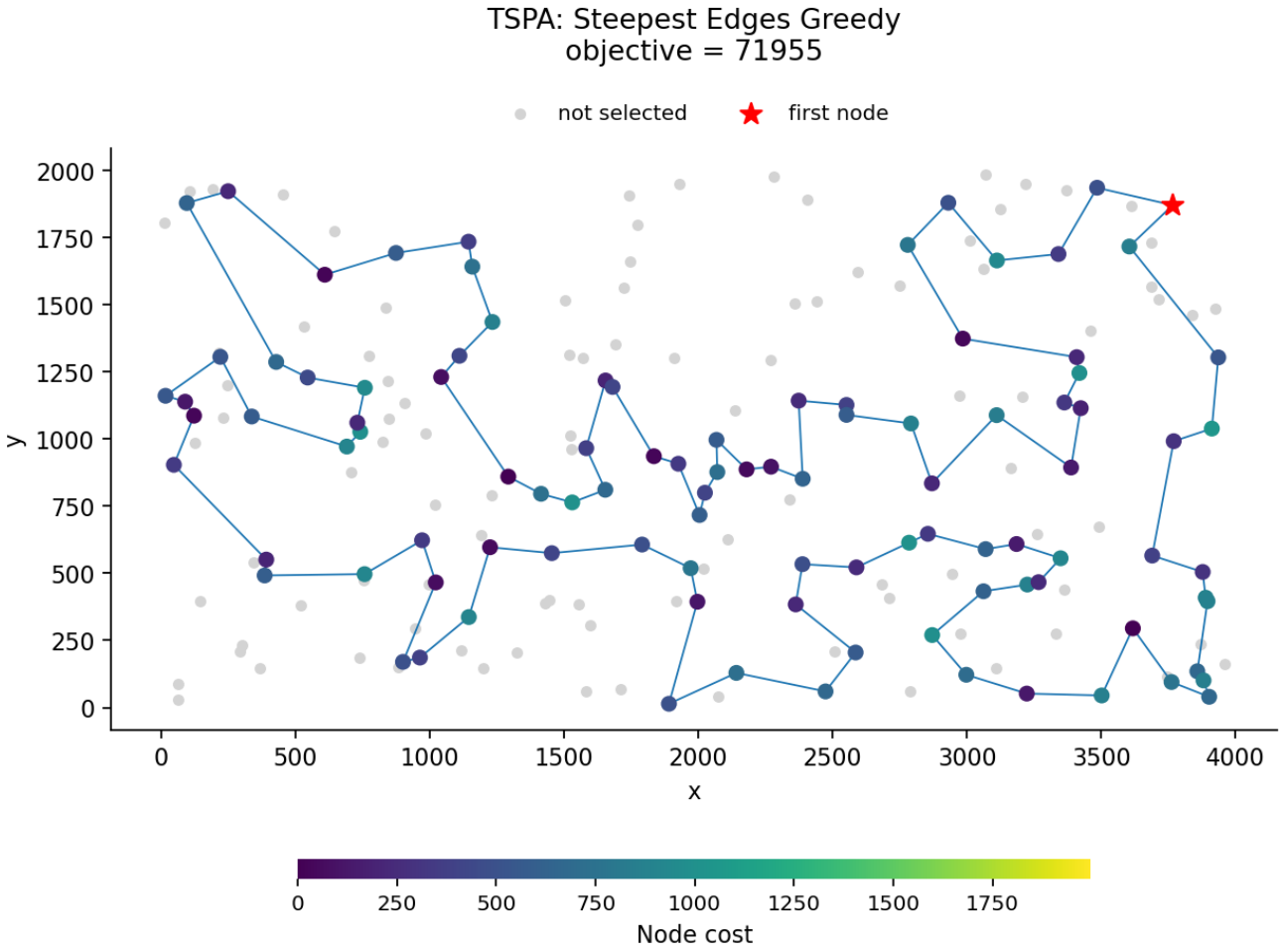


Solution (tour order):

```
 80  48  31  60 108 152  49  11  63 149
116 119 168 129 104  69 127  97 161 171
  2 189 187  30 141  87  45 140  53 179
177  36 131 134  84  58  73 135 181 192
158 182  57  64 107  88  78 153 170 159
130  47  93  43 113  89  42 151 106 150
176  72 174  85   3  50 169   1 178 101
 29 163 136 186 184 164 142   8  54   9
 33  52 144  67 185  16  51  35  28 180
160  83 123  90 132  76   4  71 157  91
```

Steepest Edges Greedy

Objective: **71955**.

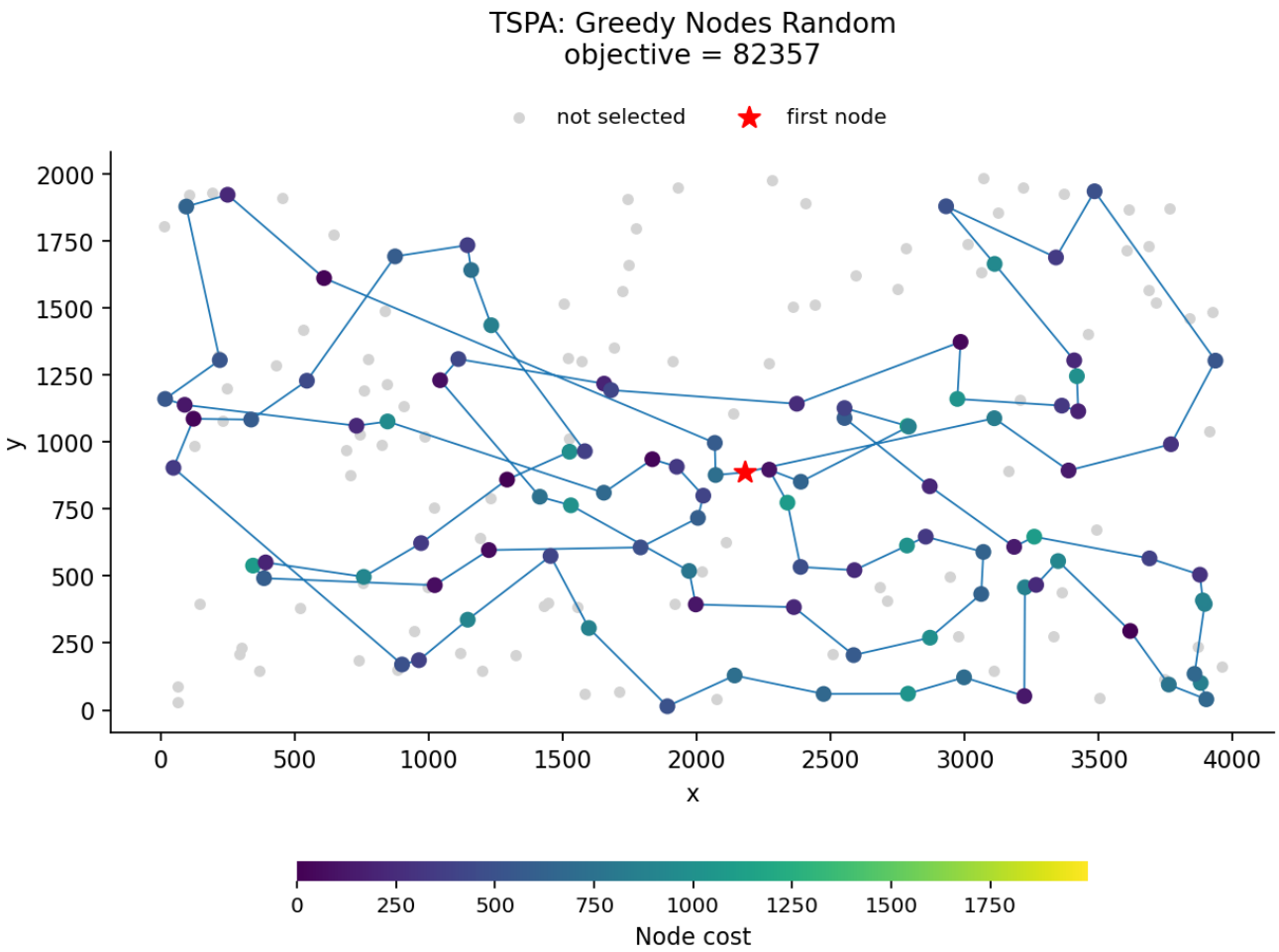


Solution (tour order):

```
 55 101 147  37  47 130 159 146 170 153
193  78  88 107  64  57 182 138 151  42
 89 113 106 150 176  72 134  84  58 158
192 181 114 135  73  45  87 141 189   2
171 187  30 161 127  97 168  63 149 116
 60  49 160  83 123 100 152 108  31  48
 80  91 157  71   4  76 132 180  28 102
185  35  51  16  67 144 140  53 179  52
177  36 131  33   9  54   8 174  85  93
 50   3 169   1 164 186 136 184 178  29
```

Greedy Nodes Random

Objective: **82357**.

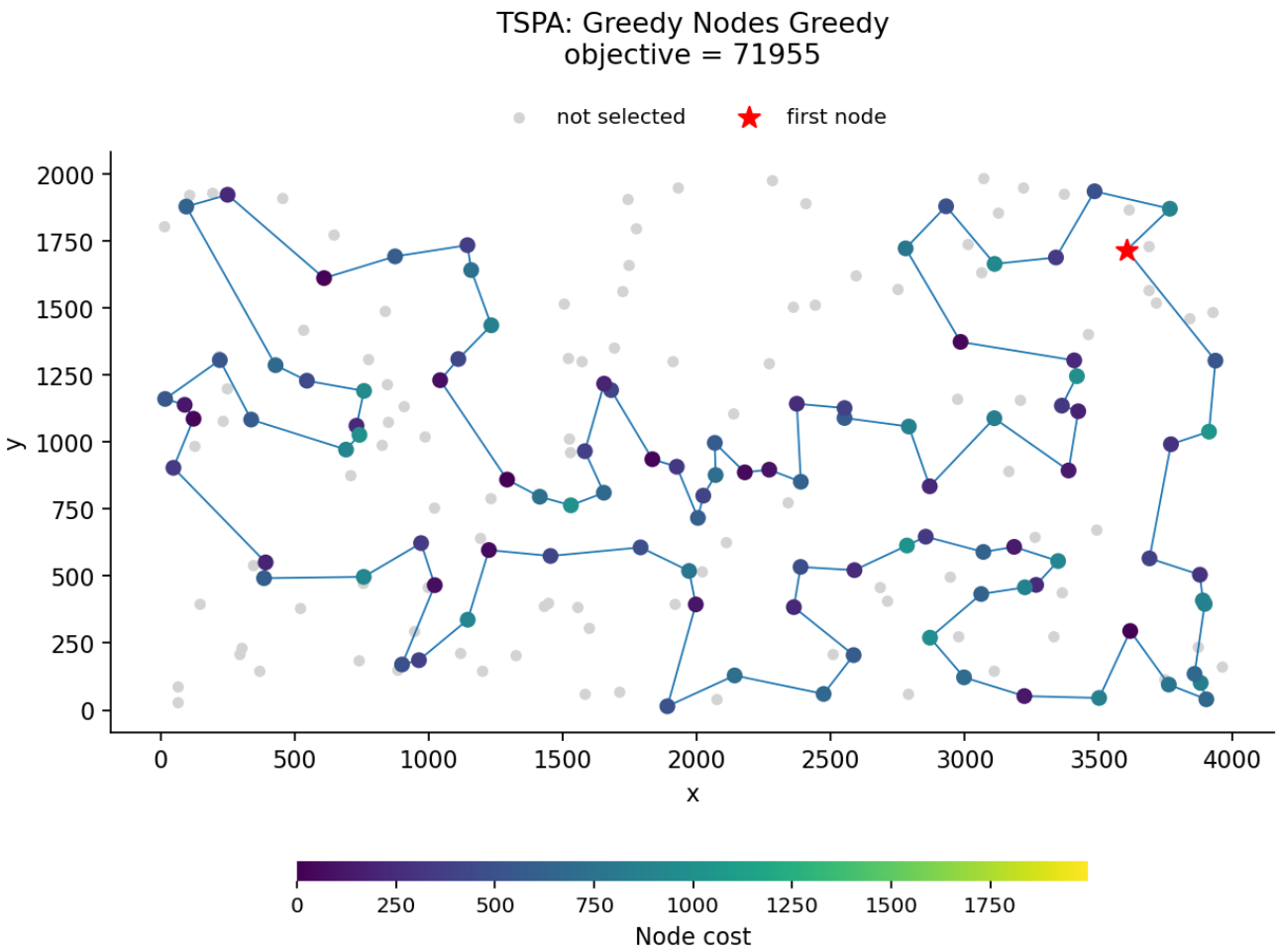


Solution (tour order):

```
177 179  52  80  48  31  60 116 149 123
 90 185  67 144  53 140  45 141 187 127
 95  97 161  30 180 156  35   4  71 157
 91 152  49  63 168 171   2 189  87  98
114 181 192 111 182  57  42  89 113 107
 88  78 193 153 170 146 159 130  43 106
174  54   9 194   8 131  36  99  84 134
 72 176 150 151 138 158  58 135  73 102
 28 132  76  51  16  33 164 142   3  50
169   1 184 136 178  29 147  47  93  85
```

Greedy Nodes Greedy

Objective: **71955**.

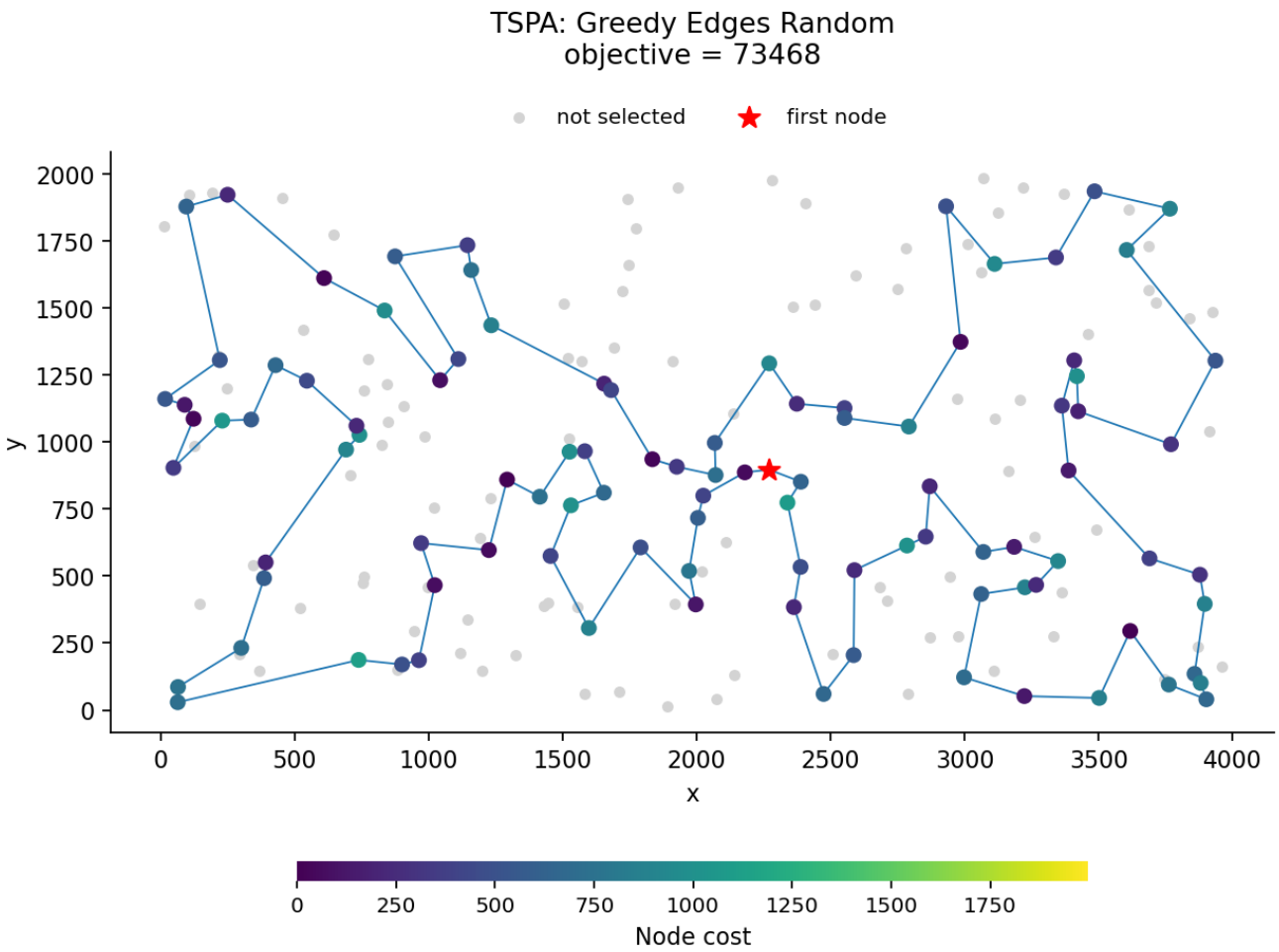


Solution (tour order):

```
101  55  29 178 184 136 186 164   1 169
  3  50  93  85 174   8  54   9  33 131
 36 177  52 179  53 140 144  67  16  51
 35 185 102  28 180 132  76   4  71 157
 91  80  48  31 108 152 100 123  83 160
 49  60 116 149  63 168  97 127 161  30
187 171   2 189 141  87  45  73 135 114
181 192 158  58  84 134  72 176 150 106
113  89  42 151 138 182  57  64 107  88
 78 193 153 170 146 159 130  47  37 147
```

Greedy Edges Random

Objective: **73468**.

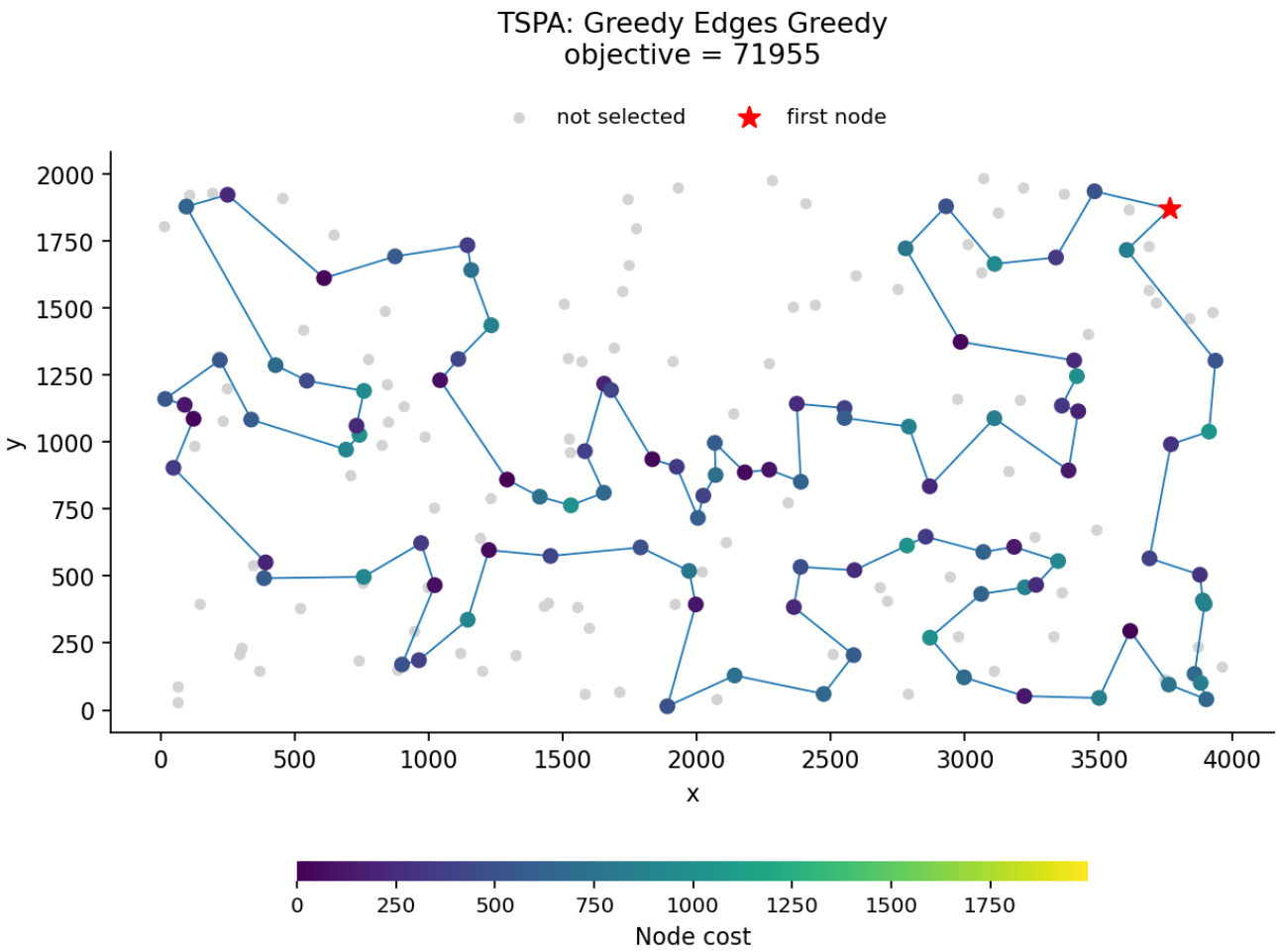


Solution (tour order):

```
 36 177  53 140  73 135  45  98  87 102
185  35 156  28 180 141  30 187   2 171
 92 104 129  69 127  97 160  83 123 152
108  49  11 168  63 149 116  60  31  48
 80  38 132  76  91 157  71   4  51  16
 67 144 179  52 143  33   9  54   8 164
136 184 178  29  55 101 147  47  50 169
  1   3  93 130 159 170 153 193  78  88
107  64  57 182 151  42  89 113 106 150
174 176  72 134 158 192  58  84  99 131
```

Greedy Edges Greedy

Objective: **71955**.



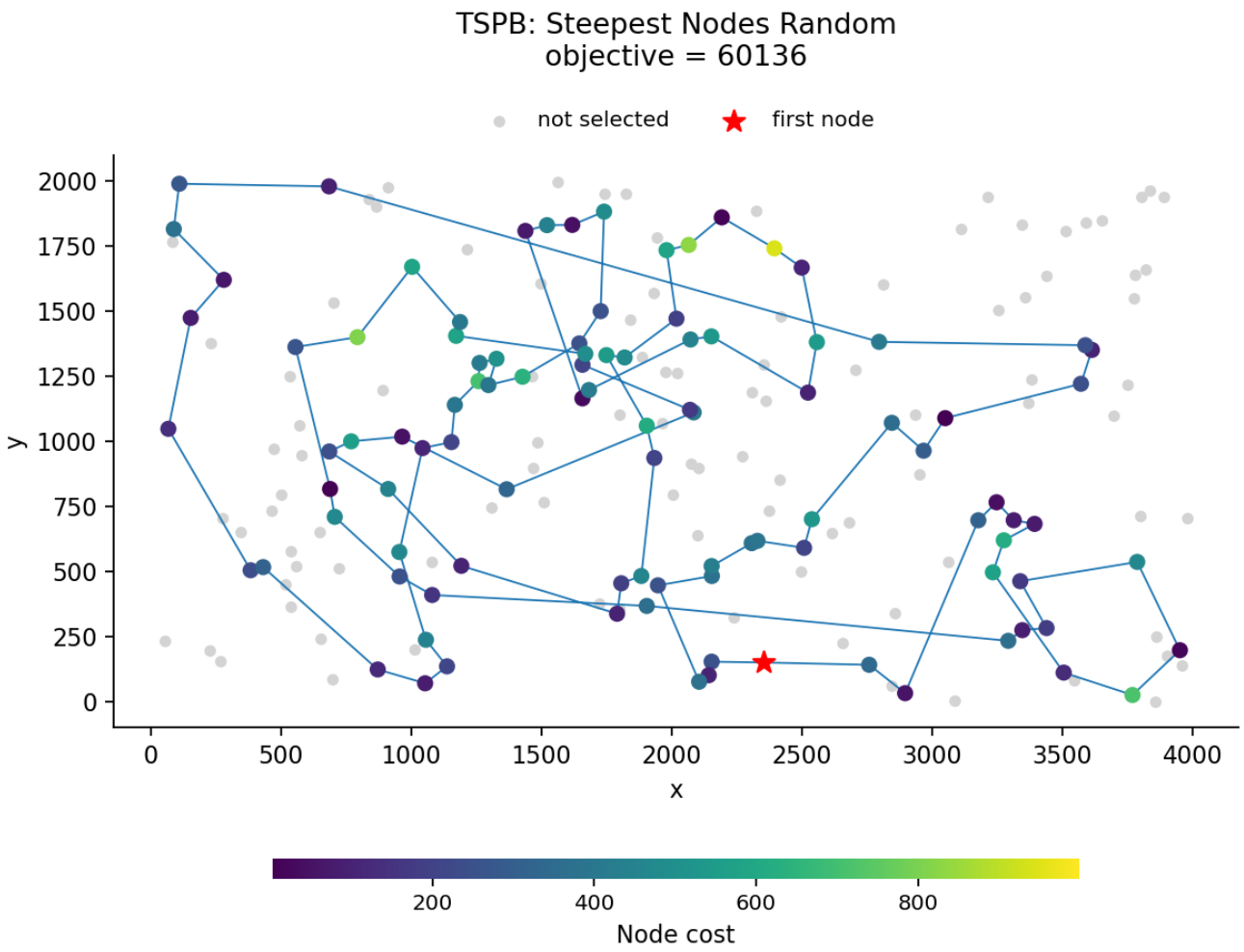
Solution (tour order):

```
 55 101 147  37  47 130 159 146 170 153
193  78  88 107  64  57 182 138 151  42
 89 113 106 150 176  72 134  84  58 158
192 181 114 135  73  45  87 141 189   2
171 187  30 161 127  97 168  63 149 116
 60  49 160  83 123 100 152 108  31  48
 80  91 157  71   4  76 132 180  28 102
185  35  51  16  67 144 140  53 179  52
177  36 131  33   9  54   8 174  85  93
 50   3 169   1 164 186 136 184 178  29
```

TSPB

Steepest Nodes Random

Objective: **60136**.

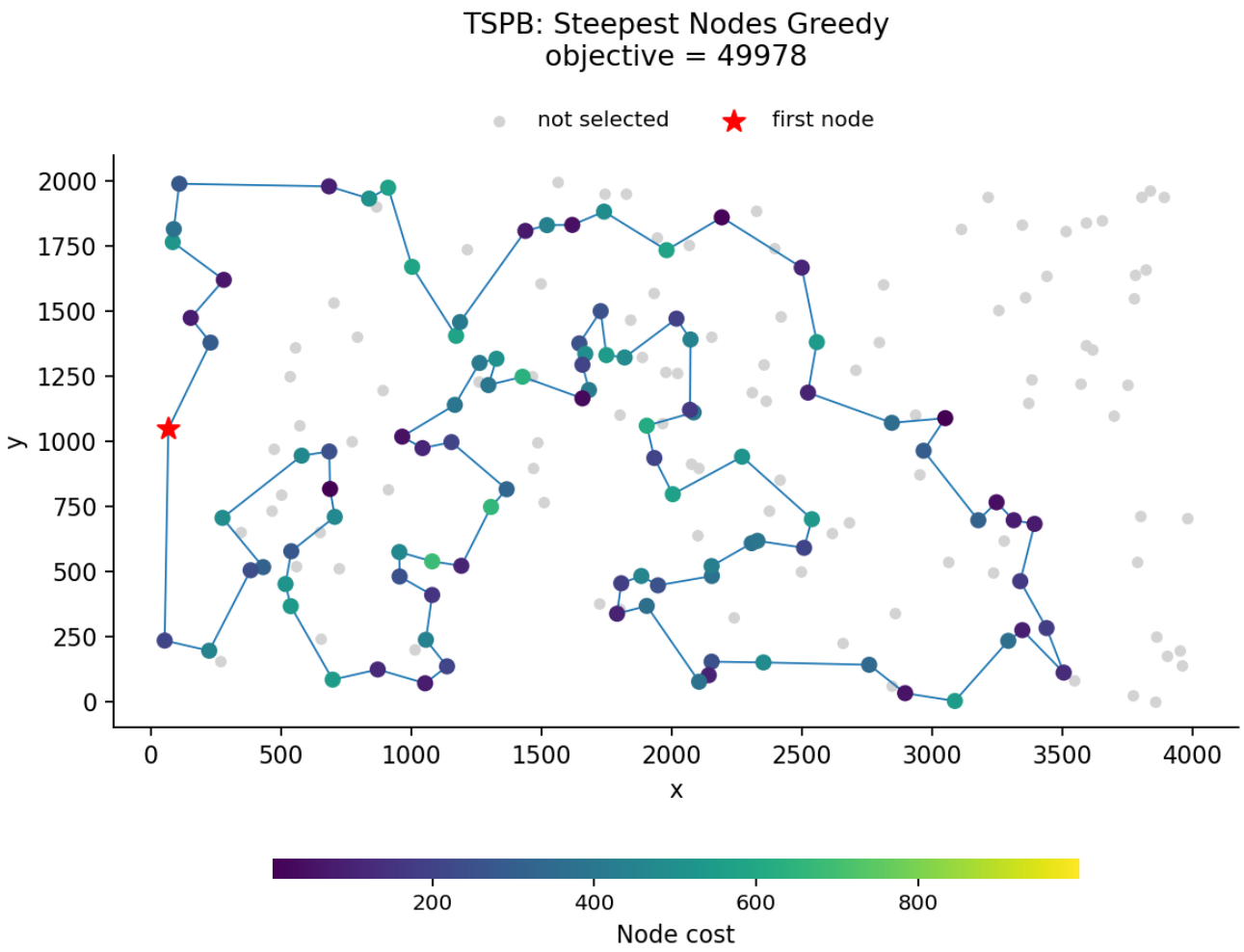


Solution (tour order):

```
 88 179  87  82 167 181 152  34  24  67
 66 162 192  54  10 114 196 104 116  37
146 149  28  65 132  29 175 134  45 157
155 140  47  38 176  30 183  21 189 133
120  48  96 198 121  69 160  46  86  76
185  57  83  89  32 156  72 117   8 122
151  73  59  85  40  97 125  26 154  64
  3 184 172 136 191 130  50  15 158 170
115  71  60  51  74  20  14 153 118 110
 93   2 182  52  90 129 127  80 171  99
```

Steepest Nodes Greedy

Objective: **49978**.

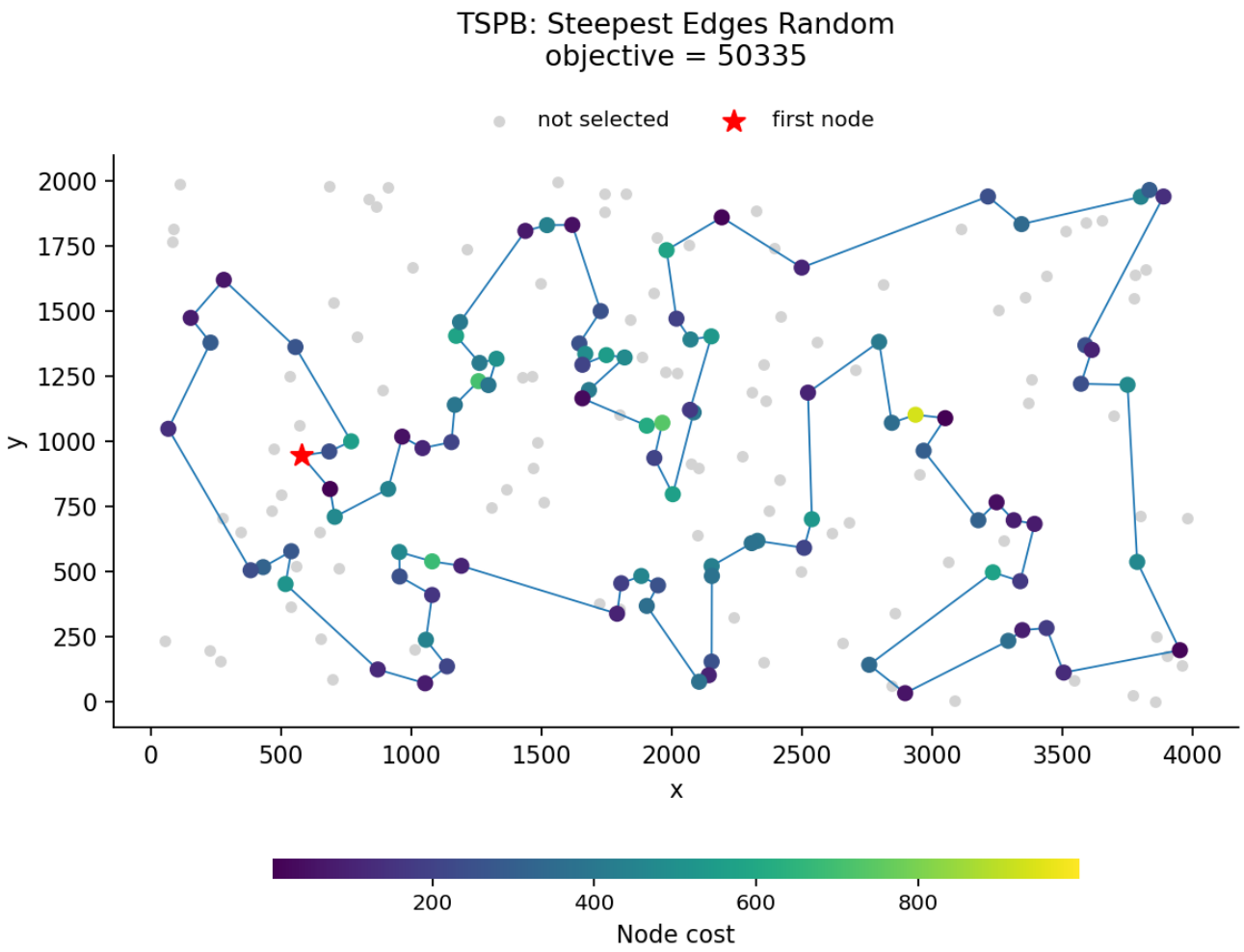


Solution (tour order):

```
 65 145  28 149  13 146  37 116 147  42
 50 191 130 121 198  96  48 156  89  57
185  76 162  54 192  80 127 129  90 153
 14   2  20  74  17 171  99  88 179  87
 82  51  85  59  73 167 181 152  34  24
 67  66  27  98 151 122   3 184  46  72
117   8 120 133 136 172 160  69 189  21
183  30  38 154 140  47  64  33  40 141
155  71  60 157  45 134 175  62  36 197
  6 115 170 125 177 178  29 132 174   1
```

Steepest Edges Random

Objective: **50335**.

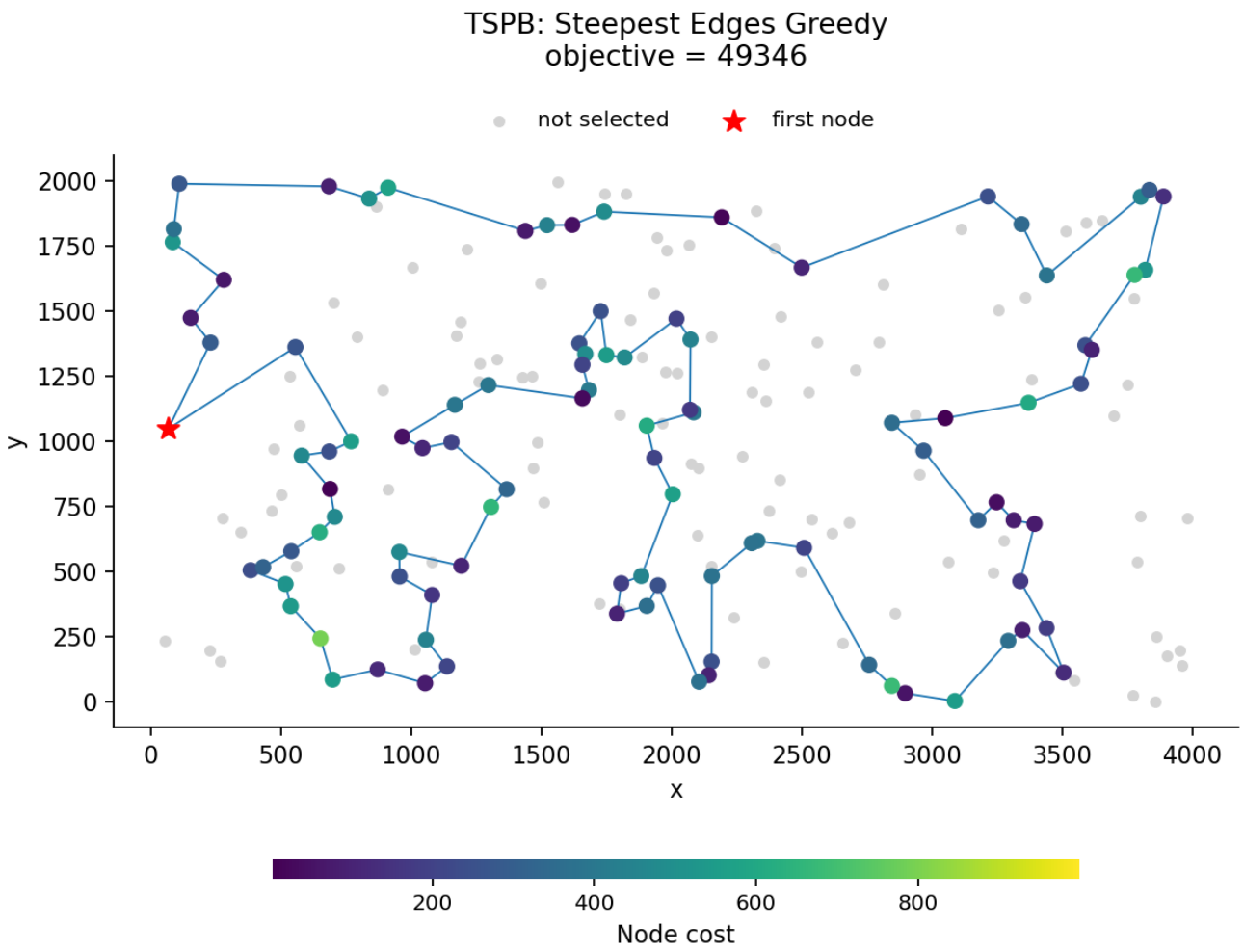


Solution (tour order):

```
177 170 115  97 154 140  47  38 176  21
183  30 191 130 121 198  96 120 133 136
172   8 117 160  69 122 173 151  98   3
184  86  46  72 156  89  57 190  70  81
 94 105 196 114  10 148 118 110   2  14
 20  74 171  99 182 153  90 129 127  80
192  54 159 162 104  76  66  67  24  34
152 181 179  87  82  51 167  73  59  85
 40 141 155  71  60 157  45 134 175 197
  6  29 132  65 145  28 149 158  26 125
```

Steepest Edges Greedy

Objective: **49346**.

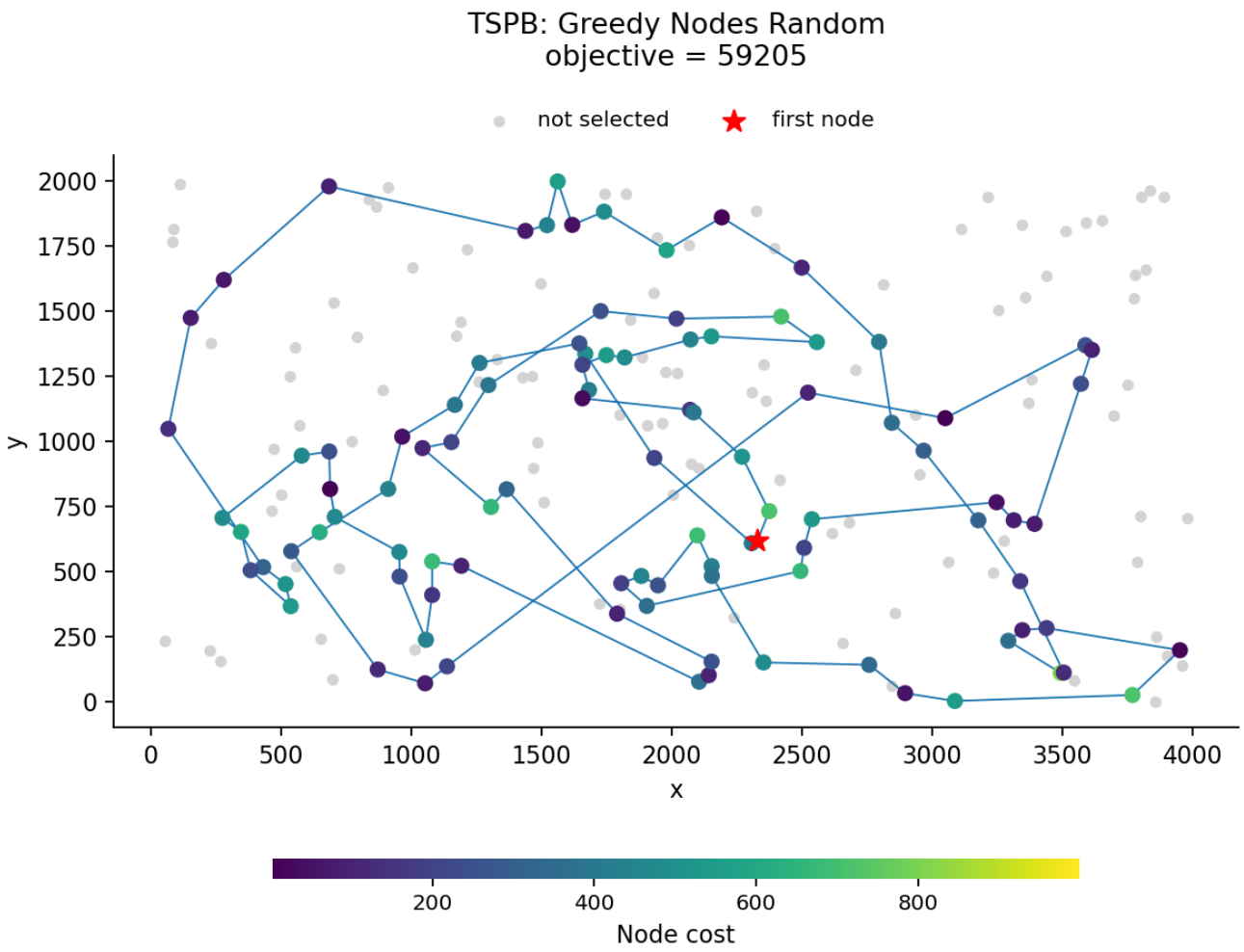


Solution (tour order):

```
 65 145  28 149  13 146  37 116 147  42
121 198  96  48  89  57 190  70 187  81
 94 105  61 139 196 114  10  78  54 162
192  80 127 129  90 153  14   2  20  74
 17 171  39  99  67  24  34 181 179  87
 82 167  51  85  59  73  98 151 122   3
184  46  72 117   8 120 133 136 172 160
 69  21  38 154 140  47  64  33  40 155
 71  60 157  45 134 175  62  91  36 197
132  29   6 180 115 170 177 125  26 158
```

Greedy Nodes Random

Objective: **59205**.

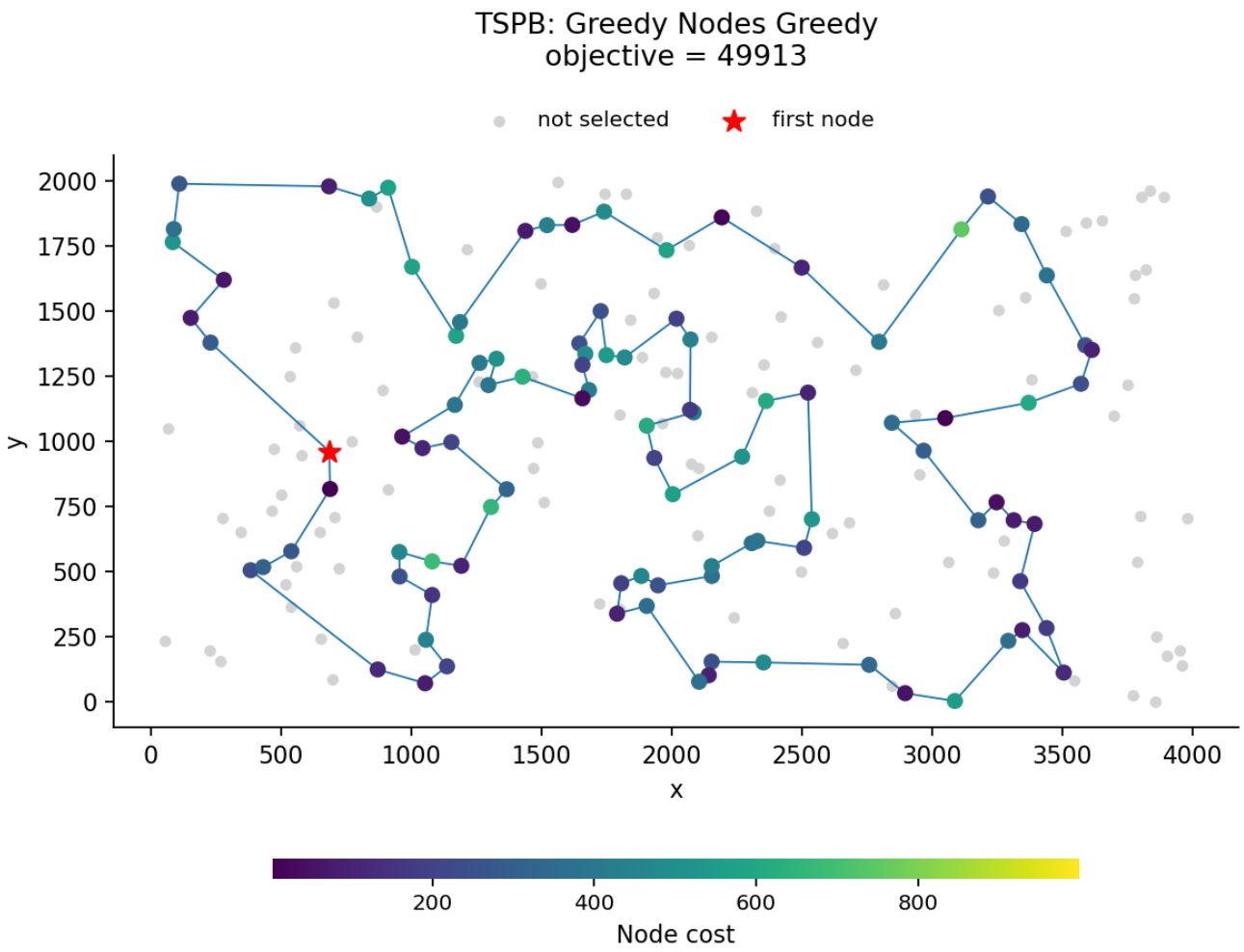


Solution (tour order):

```
 24  34 151 136 133  30  38 154  97 180
  6 175 134  45  76  54 196 114  10  90
129 127  66  67  31  51  59  73 167  25
152 181  88  99 171  17  93 110  14  20
 74 188   2 153  80 192 162 104  57  89
156  48  96  58 198 121 116 149  28  65
 29 197  36 132 103 178 177 125 170 115
155  71 157  60 141  40  82  87 179  85
 64  33 140  47  21 120  72  79 185  86
 46 117   8 172 160  69 184   3  27  11
```

Greedy Nodes Greedy

Objective: **49913**.

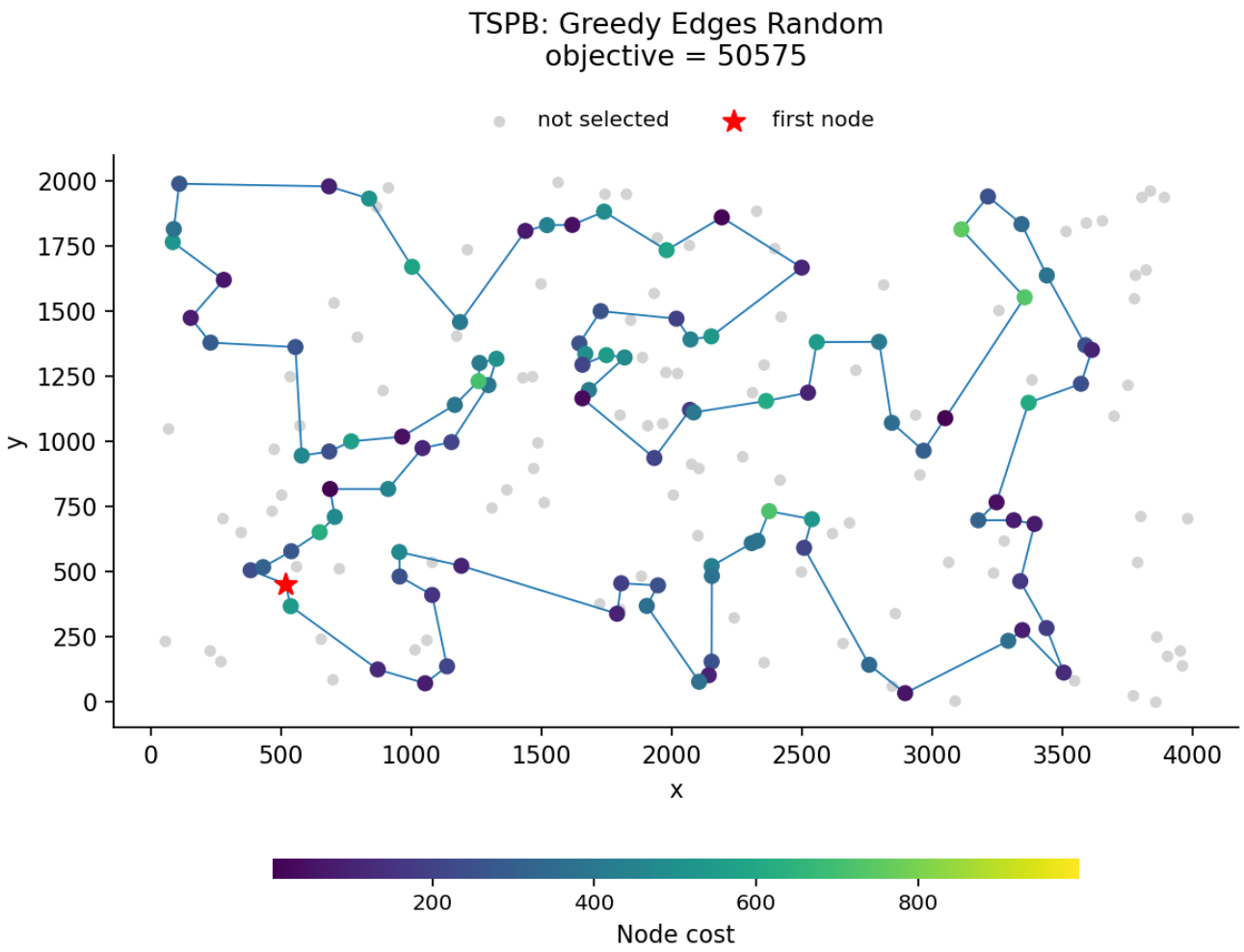


Solution (tour order):

```
125 145  28 149  13 146  37 116 147  42
 50 191 130 121 198  96  48 156  89  57
104 194 190  70 187 196 114  10  78  54
162 192  80 127 129  90 153  14   2  20
 74  17 171  99  88 179  87  82  51  85
 59  73 167 181 152  34  24  67  66  76
135  27  98 151 122   3 184  46  72 117
  8 120 133 136 172 160  69 189  21 183
 30  38 154 140  47  64  33  40 141 155
 71  60 157  45 134 175 132  29   6 170
```

Greedy Edges Random

Objective: **50575**.

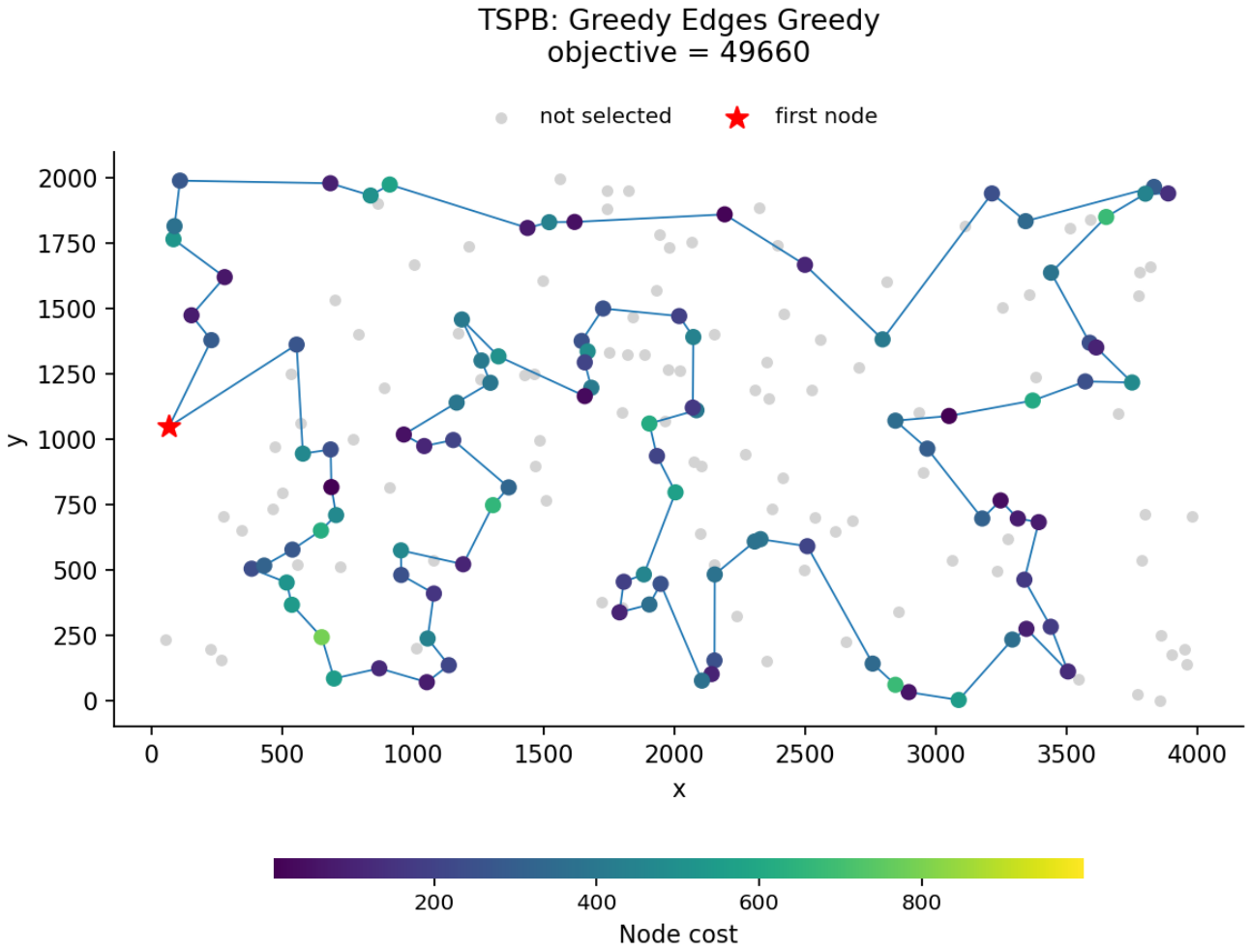


Solution (tour order):

```
197 132  29   6 180 115 170  97 140  47
 21 183  30 176  38 154  26 125 177 158
145  28 149  13 146  37 116 147  50 130
121 198  96  48 156  89  57  86  46  72
120 133 136 172   8 117 160  69 151 184
  3 135  76 185 104 162 192  54  18 194
190  70 187 196 114  10  78 127  80 129
 90 153  14   2  20  74 171  99  67  66
 11  24  34 152 181 179  87  82  51 167
 59  85  40 155  71  60  45 134 175  36
```

Greedy Edges Greedy

Objective: **49660**.



Solution (tour order):

```
 65 145  28 149  13 146  37 116 147  42
121 198  96  89  57 104 190  70  94 105
 81 169 187 196 114 148  10  78  54 162
192  80 127 129  90 153  14   2  20  74
 17 171  39  99  67  24  34 181 179  87
 82 167  51  85  59  73  98 151 122   3
184  46  72 120 133 136 172 160  69 183
130  30  21  38 154 140  47  64  33  40
155  71  60 157  45 134 175  62  91  36
197 132  29   6 180 115 170 125 177 158
```

Solution checking

All 16 best solutions were checked for exactly 100 distinct indices in the range 0-199, and their objectives match the minima in the lab 3 summary.

Source code

Repository: PiotrFoltyniewicz/Evolutionary-Computation.

- Local search (steepest, greedy, moves, starting solutions)

- Experiment runner
- Experiment statistics and timing
- Visualization script
- Exported solutions
- Lab 3 summary

Conclusions

Local search improved the greedy starting solutions. On TSPA, the best average is 72984.73 (Greedy Edges Greedy), compared with 73353.51 for the construction heuristic. All four variants with greedy starts reach the best objective of 71955. On TSPB, the best average is 52640.92 (Steepest Edges Random), while the best objective is 49346 (Steepest Edges Greedy), compared with 50485 for the construction heuristic.

Edge exchange gives better average objectives than node exchange, especially from random starts. On TSPA, random starts with edge exchange average 76.1–76.3k versus 88.3–90.3k with node exchange; on TSPB, the respective ranges are 52.6–53.0k and 63.7–65.9k. Greedy starts give similar results for both search strategies, particularly on TSPA.

From random starts, greedy search takes 24.8–28.5 ms per run, compared with 35.0–35.4 ms for steepest edge exchange and 66.7–67.7 ms for steepest node exchange. Greedy starts reduce total running times to 4.9–9.9 ms, including construction. On TSPB, random starts with edge exchange give better average objectives and smaller spreads than greedy starts, although the best individual solution comes from a greedy start.